

Task A: Secret Message Delivery

$$R = k = \log_2 N.$$

Task B: Sweepers

- A convenient way to reason about the problem is to partition the ordered set of sensors into k contiguous blocks and let one sweepers patrol each block by repeatedly walking from its leftmost to rightmost sensors and back. In such a block, the idleness equals twice the block's *span* (distance between its extreme sensors); thus minimising idleness reduces to splitting the sorted coordinates into k blocks that minimise the maximum span.
- This yields an $O(n \log n)$ solution: guess the idleness and use binary search to find the optimum idleness.

Task C: Decode Messages

Given a prime number p and an integer c , the problem is to find an integer x such that $x^2 \equiv c \pmod{p}$.

This is known as computing a square root of c modulo p . The Tonelli-Shanks algorithm provides an efficient algorithm to solve this problem when p is an odd prime. In particular, when $p \equiv 3 \pmod{4}$, the solution is considerably simpler than in the general case.

To provide sufficient mathematical background to solve the problem, the problem statement includes a specific version of the algorithm for solving the problem with $p \equiv 3 \pmod{4}$.

Task D: Bicycles

Let $h(i, f)$ be the maximum number of wheels that the first i containers can carry while also carrying f frames. It is easy to compute $h(i, f)$ for all possible f 's given the values of all $h(i - 1, f)$. The answer is the maximum f where $h(n, f) \geq 2f$.

Task E: Counting Strings

This is a typical problem from combinatorics. First we need to derive the formula and then implement the calculation carefully to beat the time limit. Given n, ℓ, k , we can derive there are $\ell(\ell - 1)^{n-k-1} \binom{n-1}{k}$ possible strings. The rest is to compute the formula efficiently. When n and ℓ are large, the formula cannot be evaluated directly and efficiently. It needs to implement calculations of the power and binomial coefficient efficiently. First let's see how to derive the above formula. Since each $1 \leq e_i \leq k + 1$ and $n = m + k$, we can determine the pattern of $s_1 \cdots s_m$ and then split the remaining k symbols into m piles. We have $\ell(\ell - 1)^{m-1}$ different possible patterns for $s_1 \cdots s_m$. There are $\binom{m+k-1}{k} = \binom{n-1}{k}$ ways to split k elements into m piles (including empty one). Thus, totally, we have $\ell(\ell - 1)^{m-1} \binom{n-1}{k} = \ell(\ell - 1)^{n-k-1} \binom{n-1}{k}$ possible strings. It is clear that powering can be computed within logarithmic iterations.

Next, we show how to compute the above binomial coefficient modulo 10^9+7 efficiently. We can use an array T of length $n+1$ to store the factorials modulo 10^9+7 in linear time, i.e., $T[p] = p! \bmod 10^9+7$ for $p = 0, \dots, n$. Note $T[p+1] = T[p] * (p+1) \bmod 10^9+7$. Then we use another table, $I[p]$, to store the inverse of $p!$ modulo 10^9+7 , which is a prime number. By the little Fermat lemma, we know the inverse of $n!$ modulo 10^9+7 is simply $(n!)^{10^9+7-2}$ modulo 10^9+7 . Note $I[p] = I[p+1] * (p+1) \bmod 10^9+7$. This shows that table I can be computed efficiently. With the tables and the above formula, we can compute the answer efficiently.

Task F: No Cycles

If G is a DAG, then all vertices in V are solutions. Assume that G contains cycles. In this case, it is not difficult to observe the following: since G is dense (i.e., for every pair of distinct vertices $u, v \in V$, there exists at least one edge between u and v), there exists a cycle C of at most three vertices. An $O(n^2)$ -time algorithm is as follows. First, find a cycle C of size 2 or 3. Since G is dense, this step can be done by a simple DFS. (Stop DFS once a back edge is found). Clearly, to make G contain no cycles, at least one of the vertices in C must be removed. Next, for each vertex x in C , check whether $G - x$ is a DAG. Since $n \leq 6000$, the above solution is feasible.

Task G: Sunshine Daily

- There are n proposed building sites, each site i is represented by (x_i, h_i) where the building spans the interval $[x_i, x_i + 4]$ along the x -axis and has h_i floors (each of height 1).
- A subset of m building sites is guaranteed to be built, while each of the remaining $n - m$ sites may or may not be built, yielding 2^{n-m} possible building plans.
- For a fixed plan, a floor X of building i is said to be *sunlit* if the total amount of time any part of the floor receives direct sunlight is last least $\alpha\%$ fraction of daylight time (the sun moves along a semicircular arc from east to west).
- A floor X in building i is called legit if there exist at least K distinct building plans such that X is sunlit in all of these K plans.
- The task is: for each i ($1 \leq i \leq n$), compute the number of legit floors in building i .

Task H: Cracking

Let the index of s_2 start from 1 and end at n . The main idea is to construct the prefix and suffix tables for s_2 . Their definitions are as follows.

1. Let $p[i]$ be the length of the longest common prefix of s_2 and $s_2[i..n]$.
2. Let $s[i]$ be the length of the longest common suffix of s_2 and $s_2[1..i]$.

These two tables can be constructed in $O(n)$ time. Without loss of generality, we assume that $|y| \geq |z|$ (defined in the problem). When $s[i] + p[i + 1] \geq i$, this indicates that $s_2[1..i]$ is one of the candidates of the answer. Symmetrically, $|y| < |z|$ can be handled similarly. Hence, the answer is the longest one. The time complexity is $O(n)$.

Task I: A Biologist's Task

Given a partial distance metric d , compute an ultrametric T such that $\max_{(u,v) \in E} |d(u,v) - d_T(u,v)|$ is minimized. First, binary search on the optimal cost Δ based on whether or not there exists an ultrametric with cost at most Δ . For each test with value δ , if there exists such a T , then

$$-\delta \leq d(u,v) - d_T(u,v) \leq \delta$$

holds for all $(u,v) \in E$, which translates to $d(u,v) - \delta \leq d_T(u,v) \leq d(u,v) + \delta$. Hence the test becomes a sandwich-type problem: Given $G_\ell = (V, E_\ell)$ with distance d_ℓ and $G_h = (V, E_h)$ with d_h , where d_ℓ and d_h are the lower-bounds and upper-bounds on the ultrametric to compute, either compute an ultrametric whose pair of distances satisfies the given lower- and upper-bounds, or to certify that no such ultrametric exists.

The answer to this question consists of two parts. First, to check if there exists such an ultrametric. For this, compute a minimum spanning forest F for G_h w.r.t. d_h . For each $u, v \in V$ such that there is a u, v -path $P_{u,v}$ in F , check if

$$d_\ell(u,v) \leq \max_{(a,b) \in P_{u,v}} d_h(a,b).$$

There is such an ultrametric if and only if this condition holds for all u, v pairs. To compute such an ultrametric when it exists, one way is to pick a pair, say, $(u,v) \in E$, with the largest lower-bound, i.e., $(u,v) := \arg \max_{(a,b) \in E} d_\ell(a,b)$. Define

$$E_{u,v} := \{(a,b) \in E_h : d_h(a,b) < d_\ell(u,v)\}$$

and consider the graph defined as $G_{u,v} = (V, E_{u,v})$. Compute an ultrametric for each of the connected components in $G_{u,v}$ recursively and join them with a root so that the height is $d_\ell(u,v)/2$. This gives a feasible ultrametric.

Task J: Twin Shields on Planet X-1012

You can perform a binary search on the shield radius, denoted as R . For each candidate value of R , enumerate all possible configurations of two zones, C_u and C_v . Construct a shield of radius R that covers both C_u and C_v . Identify all zones lying outside this shield. For these remaining zones, construct another shield using Welzl's algorithm. Finally, check whether R is feasible under this construction. By applying binary search on R , you can obtain the minimum feasible radius.

Task K: Light Sticks

We say that G contains a k -core if it has an (induced) subgraph H with minimum degree at least k . Every n -node m -edge graph G contains a $\lceil m/n \rceil$ -core. In our setting $m \approx n^{1.5}$, so G contains a $\sqrt{n}$ -core. Any $\sqrt{n}$ -node tree can be embedded into this core by greedily

growing the tree within the core. We then delete the used edges and repeat the procedure for each remaining tree on the residual graph. It takes $O(m + n)$ time to pack a tree. There are k trees, so the total runtime is $O(k(m + n)) = O(n^{2.5})$.

Task L: Viral Marketing

Let $G = (V, E)$ be an undirected graph. In round zero, activate a set $S \subseteq V$ of seeds. In each subsequent round, if more than $\deg(v)/2$ neighbors of $v \in V$ are active, activate v . It can be verified that the process can be synchronized in any other way or even asynchronously without affecting the set of active vertices at the end.

In the literature, a set S leading all vertices to being active at the end (in round one) is called an irreversible dynamic (static, respectively) monopoly. So the problem is to find an irreversible static (dynamic) monopoly of size at most $\lceil |V|/2 \rceil$ if G is cubic (non-cubic, respectively).

Finding a smallest irreversible static or dynamic monopoly is NP-hard. Finding an irreversible static monopoly of a cubic graph is easier: